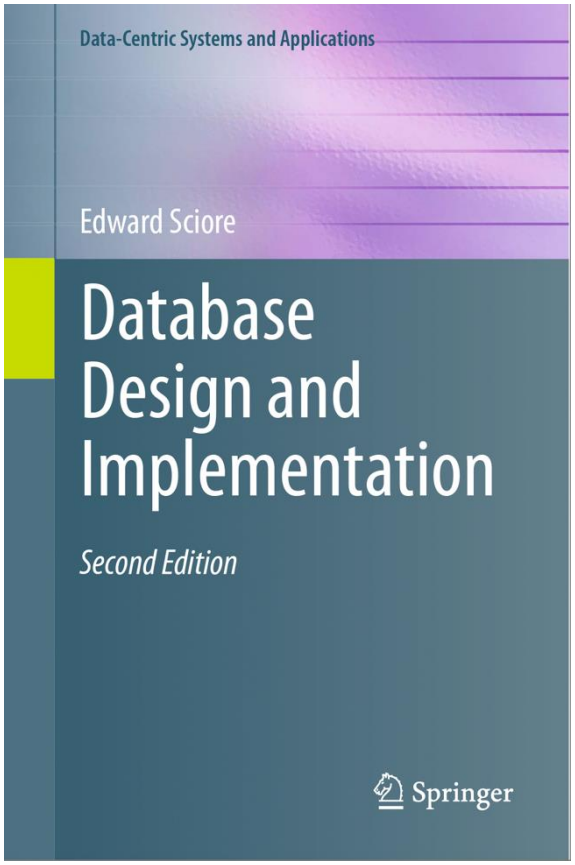
SimpleDB, un DBMS didattico

Luigi Bellomarini
luigi.bellomarini@uniroma3.it



SimpleDB

- Sviluppato da Ed Sciore, professore al Boston College come strumento didattico per comprendere la struttura di un DBMS (e insegnarla)
- Scaricabile da https://github.com/chaoyangnz/simpledb
- Versione 3.3
  - Java
  - 3500+ linee di codice
  - 157 classi
  - 15 package
- Estensioni circa 2000 altre linee di codice
- Nota: i sistemi reali open source (ad esempio Derby) sono cento volte più grandi; quelli commerciali ancora di più
- Note: The SimpleDB database system is completely unrelated to the "Amazon SimpleDB Web Service" distributed by Amazon.com

SimpleDB, installazione e primo avvio

- Download e installazione di **Java SDK.**

- Download di **SimpleDB** (file zip)
  https://github.com/chaoyangnz/simpledb

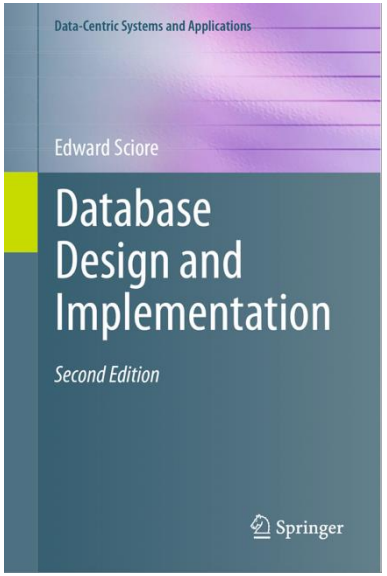
- Creare due progetti in eclipse
  - **SimpleDB3.3**
  - **SimpleClient**

- **Importare la directory simpledb** nella directory *src* di SimpleDB3.3
- **Importare il contenuto della directory simpleclient** nella directory *src* di SimpleClient.
- Impostare il build path di SimpleClient per avere visibilità del progetto SimpleDB3.3.

- Integrazione nell'IDE Eclipse
  - Tutorial (precedente versione):
    http://www.youtube.com/watch?v=spBzGSQxFI4

SimpleDB, il testo

Database Design and Implementation
Second Edition
Springer
January 2020



SimpleDB

- Un DBMS per la didattica
- Caratteristiche disponibili
  - Gestione dei file
  - Operatori di base
    - Scan
    - Filter
    - Join
    - Aggregate
  - Buffer
  - Transazioni
  - Front-end SQL
  - Parallelismo elementare

SimpleDB

- Caratteristiche non disponibili
  - Ottimizzatore di query avanzato
  - Operatori relazionali avanzati (UNION, etc)
  - Controlli di integrità
  - Tutte le caratteristiche avanzate che possono offrire i dbms commerciali

- Simple DB è un dbms prevalentemente didattico
  - Utile per apprendere la struttura di un dbms
  - Da non usare per scopi diversi

- Interessante lezione di storia dei DBMS
  - https://www.nap.edu/read/6323/chapter/8

SimpleDB, componenti

JDBC: riceve le richieste dal client e passa SQL a Planner

Planner: chiama Parser e determina piano e lo passa al Query

- Parser: analisi sintattica

Query: riceve il piano e chiama il Record per ogni tabella

- Metadata: gestisce gli schemi delle tabelle

Record: gestisce i blocchi per i record delle tabelle

Transaction: gestisce la concorrenza e l'affidabilità

Buffer: mantiene in memoria le pagine per limitare gli accessi

Log: tiene traccia delle operazioni, per l'affidabilità

File: legge e scrive le pagine su disco

Cosa si consiglia di fare

- Installare SimpleDB
- Assicurarsi del corretto funzionamento nei casi base
- Capire la struttura dei package e delle classi
- Leggere il README
- Navigare liberamente nel codice per acquisire familiarità
- Leggere la documentazione ed i javadoc

Cosa è utile saper fare

- Studio ed eventuale re-implementazione dei metodi predefiniti
  - Miglioramento del codice
  - Miglioramento delle prestazioni
- Implementazione di **nuovi metodi e nuovi algoritmi**
  - Studiati nella teoria, ma non presenti in SimpleDB
- Sviluppo di **nuove semplici funzionalità del DBMS**
- Trovare eventuali bug nel codice
- Sperimentare

Cosa potete fare

- Aggiungere nuove classi, interfacce e metodi
  - Attenti a possibili conflitti

- Aggiungere nuovi package
  - Pratica più elegante e pulita

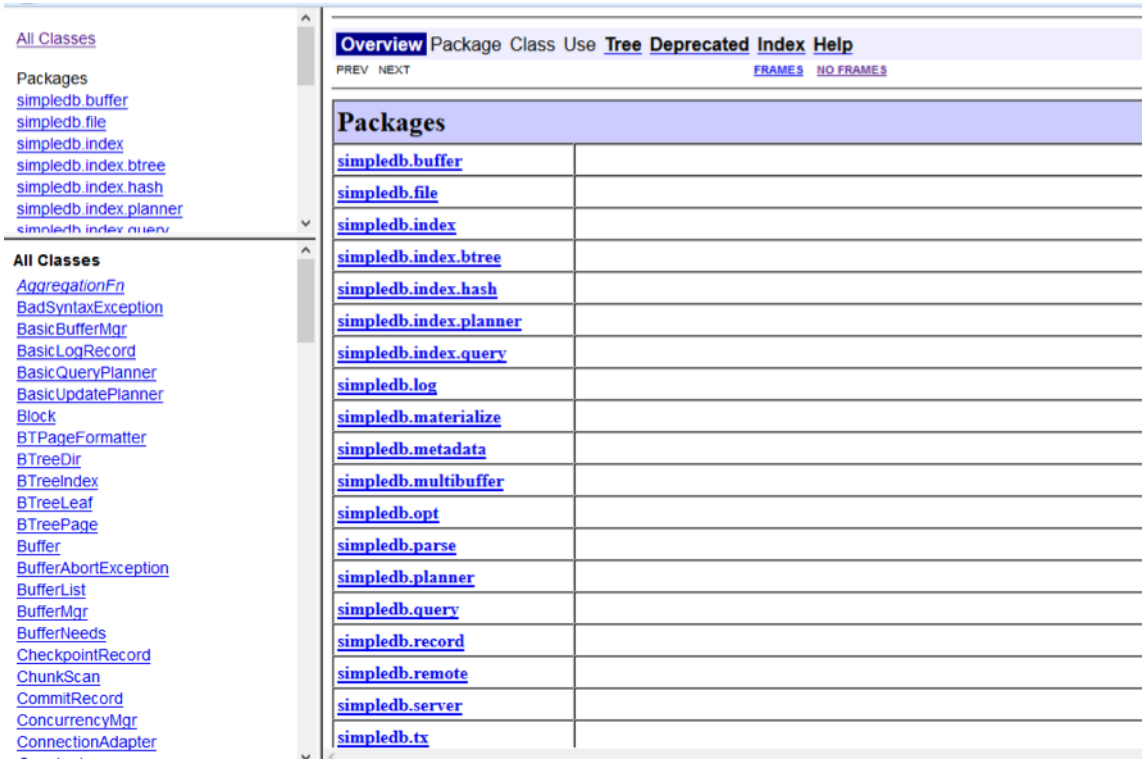
Cosa non dovete fare

- Modifiche ai nomi delle classi predefinite
  - Rimozione
  - Renaming
  - Assegnazioni ad altri package

- Modifiche ai nomi dei metodi predefiniti
  - Rimozione
  - Renaming
  - Cambio dei parametri di input
  - Cambio dei tipi e delle variabili di ritorno

JAVADOC

- SimpleDB dispone di un'esaustiva documentazione javadoc
- consultatela per effettuare le vostre sperimentazioni



Esercitazioni pratiche

- SimpleDB è fatto per essere modificato
- Prima ancora testiamo le classi nei singoli package
- Allo scopo, due avvertenze:
  - spesso le classi di test debbono poter accedere a metodi a livello di package ("friend") e quindi è spesso necessario **scrivere classi di test nei package**
  - per testare i moduli di livello basso, non serve lanciare il server, ma è **necessario chiamare alcuni metodi che costruiscono oggetti fondamentali del server** (ad esempio il FileMgr o il BufferMgr)

Parte 1

SimpleDB, componenti "read-only"

JDBC: riceve le richieste dal client e passa SQL a Planner

Planner: chiama Parser e determina piano e lo passa al Query

Parser: analisi sintattica

Query: riceve il piano e chiama il Record per ogni tabella

Metadata: gestisce gli schemi delle tabelle

Record: gestisce i blocchi per i record delle tabelle

Transaction: gestisce la concorrenza e l'affidabilità

Buffer: mantiene in memoria le pagine per limitare gli accessi

Log: tiene traccia delle operazioni, per l'affidabilità

File: legge e scrive le pagine su disco

Un'idea del funzionamento

- Un client JDBC sottomette una SELECT o DELETE o UPDATE scritta in SQL attraverso un metodo executeUpdate(…)
  - JDBC riceve la chiamata e passa a Planner la stringa SQL
  - Planner invia la stringa a Parser che restituisce i nomi delle tabelle e le condizioni; con ciò, Planner può costruire un piano di esecuzione (espressione algebrica) che invia a Query
  - Query crea uno scan per ogni operazione nel piano
  - Record gestisce i record conoscendone l'organizzazione in blocchi e quindi richiede a Transaction la lettura (o scrittura) di specifici dati
  - Transaction, se non ci sono problemi di concorrenza chiede a Buffer di leggere o scrivere
  - Buffer (coordinandosi con Log) stabilisce quali operazioni di lettura e scrittura servono davvero e le richiede a File
  - File legge e scrive (da disco verso pagina e viceversa)

SimpleDB, componenti "read-only"

JDBC: riceve le richieste dal client e passa SQL a Planner

Planner: chiama Parser e determina piano e lo passa al Query

Parser: analisi sintattica

Query: riceve il piano e chiama il Record per ogni tabella

Metadata: gestisce gli schemi delle tabelle

Record: gestisce i blocchi per i record delle tabelle

Transaction: gestisce la concorrenza e l'affidabilità

Buffer: mantiene in memoria le pagine per limitare gli accessi

Log: tiene traccia delle operazioni, per l'affidabilità

File: legge e scrive le pagine su disco

File Management

- A database system stores its data on physical devices such as disks, e.g., Solid State Drives
- Operating system provides for interacting with these devices – a **block-level interface** and a **file-level interface**
- A database needs persistent data storage
- **Disk drives** and **flash drives** are common examples of persistent storage devices
- The general performance of a disk drive can be measured by four values:
  - *capacity*
  - *rotation speed*
  - *transfer rate*
  - *average seek time*

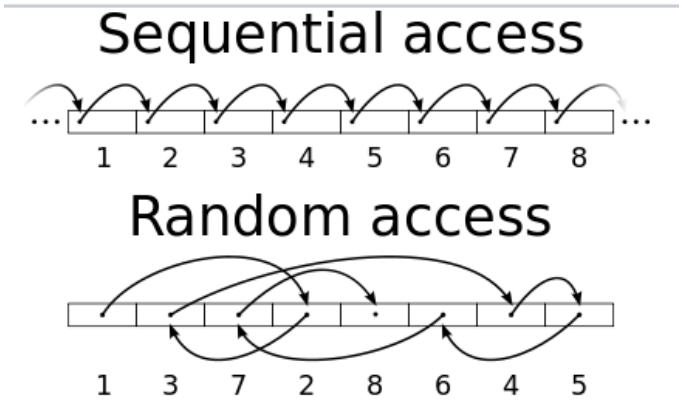
File-level interface to the disk

- The OS provides another, **higher-level interface to the disk**
  - The **file system**
- A client views files as named sequences of bytes.
- **Disk blocks** must be accessed through **pages**.
- The OS hides this, and the disk is directly accessible to the user, apparently.
- OS typically reserves memory for several pages, I/O buffers.
- Logical and physical blocks.

- **Types of allocation**
  - Continuous
  - Extent-based
  - Indexed

File-level interface to the disk, 2

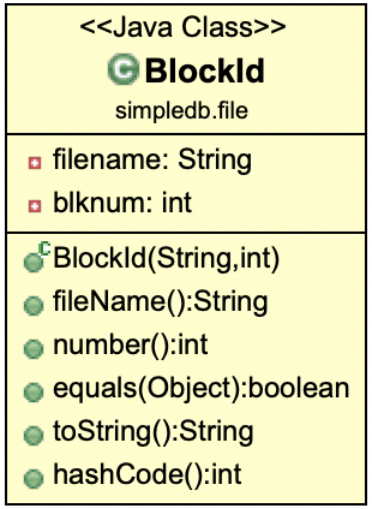
- The Java class *RandomAccessFile* provides a typical API to the file system. Each *RandomAccessFile* object holds a *file pointer* that indicates the byte at which the next read or write operation will occur. This file pointer can be set explicitly by a call to *seek*.
- A call to *readInt* (or *writeInt*) will also move the file pointer past the integer it read (or wrote)
- *The seek method performs two conversions:*
  - *It converts the specified byte position to a **logical block reference**;*
  - *It converts the logical block reference to a **physical block reference**.*

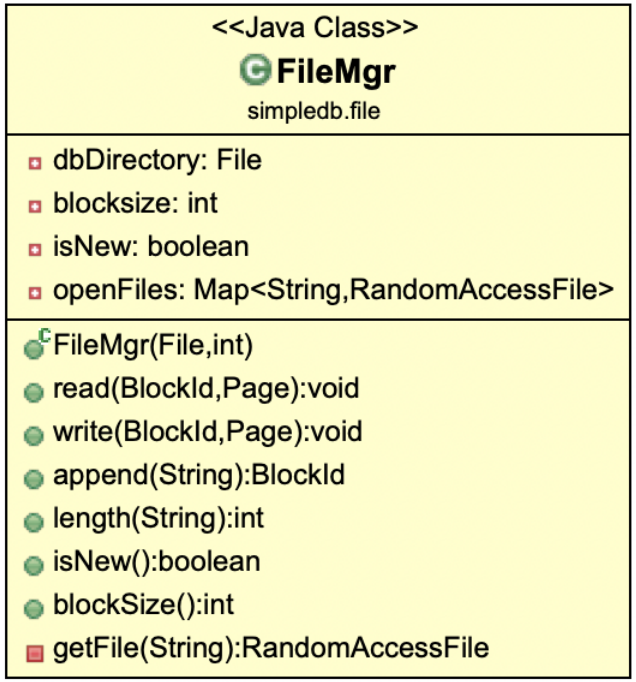


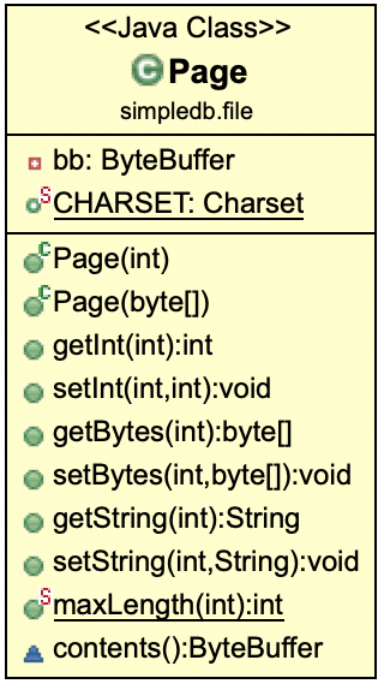
File Manager, API

```
FileMgr
    dbDirectory : File
    blocksize : int
    isNew : boolean
    openFiles : Map<String, RandomAccessFile>
    FileMgr(File, int)
    read(BlockId, Page) : void
    write(BlockId, Page) : void
    append(String) : BlockId
    length(String) : int
    isNew() : boolean
    blockSize() : int
    getFile(String) : RandomAccessFile
```

Package file







Esercizio pratico 1.1: implementare statistiche di accesso a livello di blocco (1/2)

- **Creare una classe per memorizzare le statistiche di lettura/scrittura di blocchi, ad es: simpledb.file.BlockStats**
  - La classe dovrà memorizzare il numero di blocchi letti e scritti suddividendoli per file

- **Metodi di BlockStats:**
  - toString() – che stampa la statistiche
  - logReadBlock(Block) – che aggiorna le statistiche sulla base della lettura di un blocco
  - logWrittenBlock(Block) – che aggiorna le statistiche sulla base della scrittura di un blocco
  - reset() – che azzera le statistiche memorizzate fino a quel momento

Esercizio pratico: implementare statistiche di accesso a livello di blocco (2/2)

- **Modificare la classe FileMgr, in modo tale che implementi metodi per accedere alle statistiche sui blocchi.**

- **Metodi di FileMgr:**
  - getBlockStats() – che ritorna un'istanza di BlockStats
  - resetBlockStats() – che azzera le statistiche memorizzate fino a quel momento

- Nel package JDBC, modificare il metodo commit() di EmbeddedConnection per stampare le statistiche sui blocchi al commit di ogni transazione.